Classical vs. Quantum Random Walks

Julia Huynh

Classical Random Walks

- Imagine you start at rest and move based off a coin flip (p = ½)

- If you get heads, move left. Tails, move right.

- After N steps done over many samples, we get the normal distribution (implication: the average final position is around where you started)

- Standard Deviation: $\sigma \propto \text{sqrt}(N)$

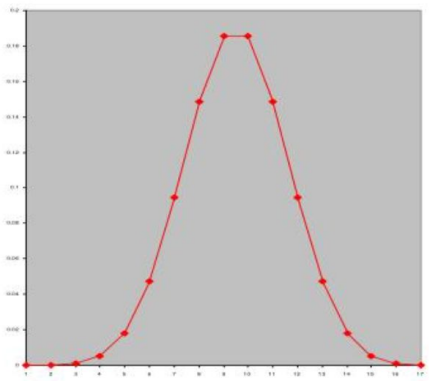
Applications (Including but not limited to):

Physics: Diffusion, Brownian Motion

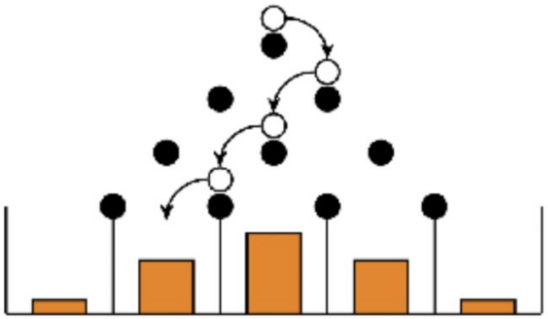
Computer Science: Random Algorithms, PageRank

Math: Markov Chains, Probability theory

Finance: Stock Price Modeling, Option Pricing



Distance: $\Theta(\sqrt{t})$



Classical Random Walk

Quantum Random Walks

- Now we still have a coin, but imagine the coin is in superposition of heads or tails

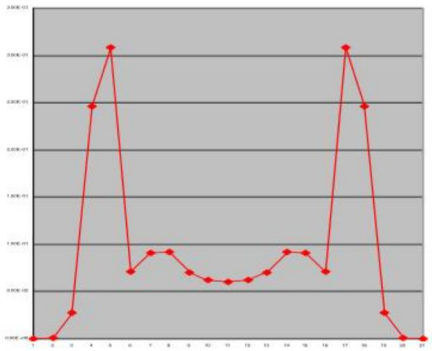
- That means every step we take has a probability of going left or right

- As we take more steps, the probability amplitudes interfere constructively or destructively. Eventually when the steps are over, we measure the state to collapse it to a final position based off that probability distribution.

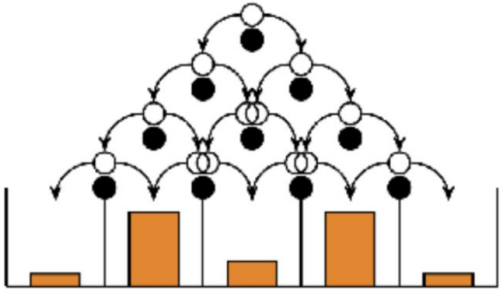
- Standard Deviation: $\sigma \propto N$

Applications:

Quantum computing & quantum information science algorithms in cryptography, machine learning, etc.

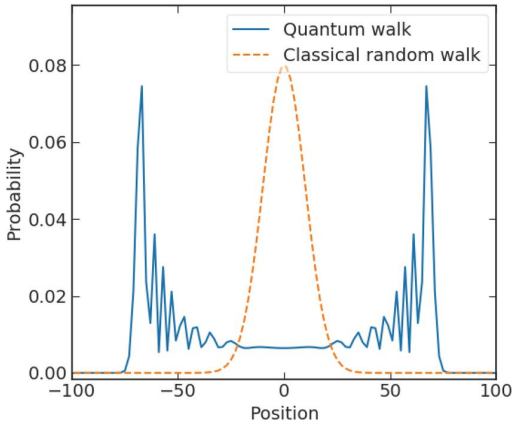


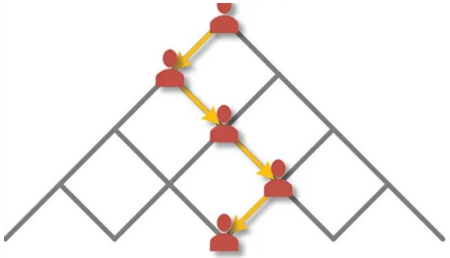
Distance: $\Theta(t)$



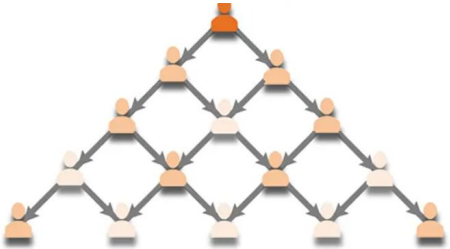
Quantum Walk

Feature	Classical Random Walk	Quantum Random Walk
Position Distribution	Gaussian	Non-Gaussian with multiple peaks
Mean Displacement $\langle x_n \rangle$	0	0
Variance $\langle x_n^2 \rangle$	Linear growth ($\sim n$)	Quadratic growth ($\sim n^2$)
Speed of Spread	Diffusive ($\sim \sqrt{n}$)	Ballistic ($\sim n$)
Interference Effects	None	Strong constructive/destructive interference
Applications	Diffusion, Brownian motion, random walks	Quantum algorithms, quantum search, quantum diffusion





(a) Random walk



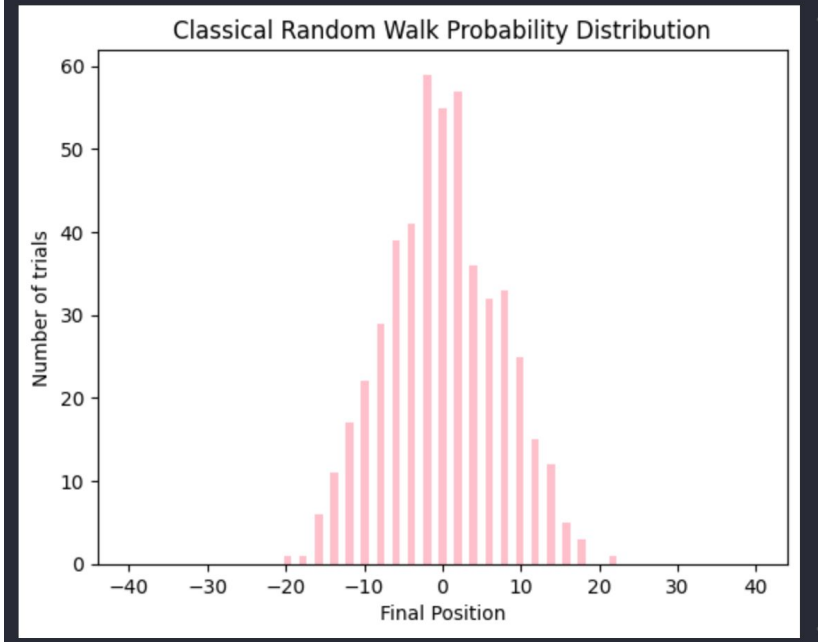
(b) Quantum walk

Conclusion: quantum random walks are pretty cool!

Coding the Classical Random Walk

```python
# Classical Random Walk Simulator

# Initialize steps and samples
n_steps = 50
n_samples = 500

# Simulate the random walk
final_positions = []
for sample in range(n_samples):
    position = 0
    for step in range(n_steps):
        position += random.choice([-1, 1])
    final_positions.append(position)

# Calculate & print the standard deviation
standard_dev = np.std(final_positions)
print(f" Standard Deviation: {standard_dev}")

# Plot the probability distribution
plt.hist(final_positions, range=(-40, 40), bins=81, color="pink")
plt.title('Classical Random Walk Probability Distribution')
plt.xlabel('Final Position')
plt.ylabel('Number of trials')
plt.show()
```

```
Standard Deviation: 7.352805178977612
```



Coding the Quantum Random Walk

Hadamard Gate: Puts a state into superposition

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

1. Apply the Hadamard to the coin state

$$H|\uparrow\rangle = \frac{1}{\sqrt{2}}(|\uparrow\rangle + |\downarrow\rangle)$$

2. Take tensor product of the coin & position state

$$|\psi\rangle = \frac{1}{\sqrt{2}}\left(|\uparrow\rangle \otimes |0\rangle + |\downarrow\rangle \otimes |0\rangle\right),$$

$$\begin{bmatrix} x_0 \\ x_1 \end{bmatrix} \otimes \begin{bmatrix} y_0 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_0 \begin{bmatrix} y_0 \\ y_1 \end{bmatrix} \\ x_1 \begin{bmatrix} y_0 \\ y_1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} x_0 y_0 \\ x_0 y_1 \\ x_1 y_0 \\ x_1 y_1 \end{bmatrix}$$

```python
# Quantum Random Walk Simulator

# Parameter Initialization
positions = 91 # Necessary to determine the size of the position statevector
steps = 50
samples = 500
center = positions // 2

# Create the Hadamard Gate for superposition
H = (1/np.sqrt(2)) * np.array([[1, 1],[1, -1]])

def run_single_walk():
    # Create the coin state of the amplitude of each side of the coin
    coin_state = (1/np.sqrt(2)) * np.array([1, 1j])

    # Create the position state of the amplitude of each position
    position_state = np.zeros(positions)
    position_state[center] = 1  # Sets the center to 100% probability

    # Statevector of the amplitudes of each coin side at each specific position
    state = np.kron(coin_state, position_state)
```

3. Apply the Shift Operator to update positions based on the side of the coin for each step

$$S = |\uparrow\rangle\langle\uparrow| \otimes \sum_i |i+1\rangle\langle i| + |\downarrow\rangle\langle\downarrow| \otimes \sum_i |i-1\rangle\langle i|.$$

$$S|\psi\rangle = \frac{1}{\sqrt{2}}(|\uparrow\rangle \otimes |1\rangle + |\downarrow\rangle \otimes |-1\rangle)$$

4. Square the amplitudes to get probabilities and add the coin sides together to get a probability distribution

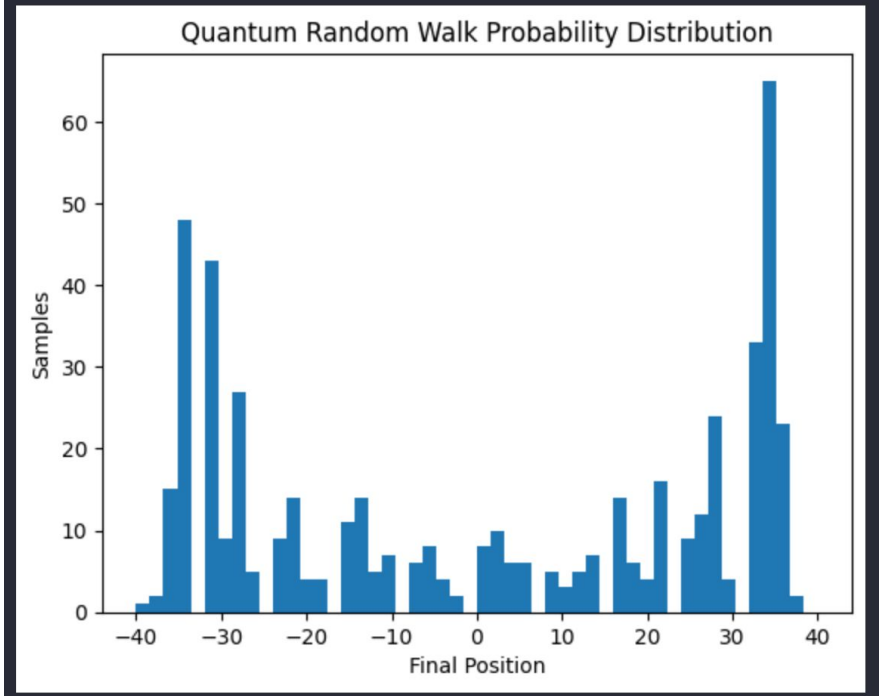
$$\text{Probability}(x) = |\text{amplitude}(x)|^2.$$

Note: since we've been using unitary matrices, the probabilities are already normalized!

5. Randomly choose a position based off those probabilities to "collapse the state into a final position"

```python
# Apply the shift operator
for step in range(steps):
    new_state = np.zeros_like(state, dtype=complex)

    for pos in range(positions):
        for coin in [0, 1]: # 0 as heads, 1 as tails
            index = coin * positions + pos
            amp = state[index]

            if abs(amp) > 0:
                for new_coin in [0, 1]:
                    # Gives how much of the coin flows into the new coin state after H
                    coin_amp = H[new_coin, coin] * amp

                    # 0 is moving left, 1 is moving right
                    if new_coin == 0 and pos > 0:
                        new_index = new_coin * positions + (pos - 1)
                    elif new_coin == 1 and pos < positions - 1:
                        new_index = new_coin * positions + (pos + 1)
                    new_state[new_index] += coin_amp
    state = new_state

# Add the probabilities of the state vector together by squaring the amplitude
# Note: the probabilities when summed, should equal 1 to be normalized!
prob = np.zeros(positions)
for pos in range(positions):
    for coin in [0, 1]:
        index = coin * positions + pos
        prob[pos] += abs(state[index])**2

# Measure the system to collapse its state
return np.random.choice(range(positions), p=prob)
```

```
Standard Deviation: 27.33315173923417
```

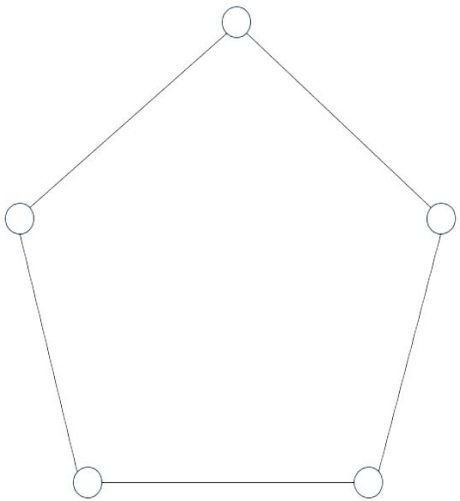


- Initial state: $|\uparrow 0\rangle$
- Step 1: $\frac{1}{\sqrt{2}}(|\uparrow 1\rangle + |\downarrow -1\rangle)$
- Step 2: $\frac{1}{2}(|\uparrow 2\rangle + |\downarrow 0\rangle + |\uparrow 0\rangle - |\downarrow -2\rangle)$
- Step 3: $\frac{1}{2\sqrt{2}}(|\uparrow 3\rangle + |\downarrow 1\rangle + |\uparrow 1\rangle - |\downarrow -1\rangle + |\uparrow 1\rangle + |\downarrow -1\rangle - |\uparrow -1\rangle + |\downarrow -3\rangle) = \frac{1}{2\sqrt{2}}(|\uparrow 3\rangle + |\downarrow 1\rangle + 2|\uparrow 1\rangle - |\uparrow -1\rangle + |\downarrow -3\rangle)$
- Step 4: $\frac{1}{4}(|\uparrow 4\rangle + |\downarrow 2\rangle + |\uparrow 2\rangle - |\downarrow 0\rangle + |\uparrow 2\rangle + |\downarrow 0\rangle + |\uparrow 2\rangle + |\downarrow 0\rangle - |\uparrow 0\rangle - |\downarrow -2\rangle + |\uparrow -2\rangle - |\downarrow -4\rangle) = \frac{1}{4}(|\uparrow 4\rangle + |\downarrow 2\rangle + 3|\uparrow 2\rangle + |\downarrow 0\rangle - |\uparrow 0\rangle - |\downarrow -2\rangle + |\uparrow -2\rangle - |\downarrow -4\rangle)$

```python
# Run multiple samples
results = []
for sample in range(samples):
    final_pos = run_single_walk()
    results.append(final_pos - center)

# Calculate the standard deviation
standard_dev = np.std(results)
print(f" Standard Deviation: {standard_dev}")

# Plot the graph using Matplotlib
plt.hist(results, range=(-40, 40), bins=50)
plt.title("Quantum Random Walk Probability Distribution")
plt.xlabel("Final Position")
plt.ylabel("Samples")
plt.show()
```

Classical Random Walk Search

If we had a ring with N nodes and randomly choose a target node, how long will it take for a classical random walk to find that target node?



```python
# The classical RW runtime for searching for a target node on a 1D ring
def classical_runtime(n_nodes, samples=500):
    runtimes = []
    target = random.randint(0, n_nodes - 1) # Randomly chooses a target

    # Perform many samples
    for sample in range(samples):
        # Initialize position and steps
        position = 0
        steps = 0

        # Randomly move until we reach the target node
        while position != target:
            movement = random.choice([-1, 1])
            position = (position + movement) % n_nodes # Update the position
            steps += 1 # Increase steps every movement
        runtimes.append(steps)

    return runtimes, np.mean(runtimes)
```

Quantum Random Walk Search

Same as before:

1. Superimpose coin
2. Tensor product coin & position state
3. Apply the Shift Operator
4. Square amplitudes and add to get the probability distribution
5. Measure

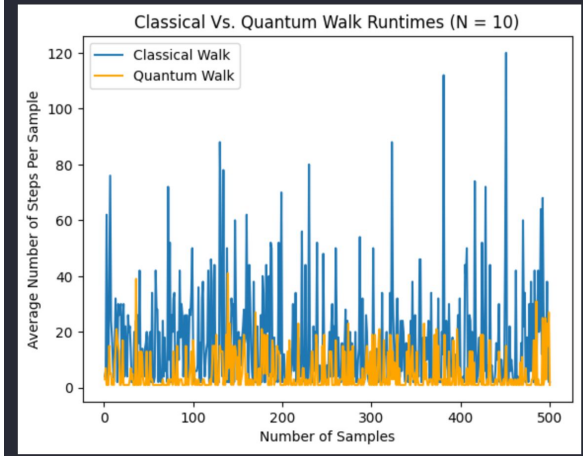
Difference:

- We don't know how many steps, so we run a while loop and add 1 step until we reach the target (measurement = target)

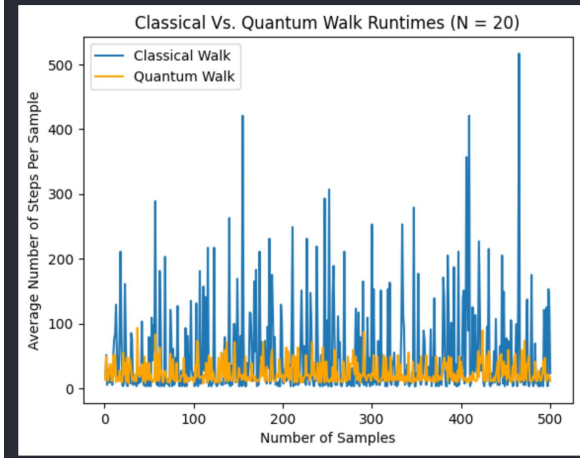
```python
# The quantum RW runtime for searching for a target node on a 1D ring
def quantum_runtime(n_nodes, samples=500):
    runtimes = []
    target = random.randint(0, n_nodes - 1) # Randomly select a target node
    H = 1/np.sqrt(2) * np.array([[1, 1], [1, -1]]) # Initiate the Hadamard gate matrix

    for sample in range(samples):
        # Initialize the start position & steps
        start_position = 0
        steps = 0

        # Create the coin and position statevectors
        coin_state = (1/np.sqrt(2)) * np.array([1, 1j])
        position_state = np.zeros(n_nodes)
        position_state[start_position] = 1 # Sets probability to 100% at start

        # Take the tensor product of the position state and coin state
        state = np.kron(coin_state, position_state)
        measurement = -1   # Initialize a measurement variable to end the loop

        # End the while loop when the measured position is at the target node
        while measurement != target:
            steps += 1 # Add a step every movement taken

            # Calculate the coin amplitude and apply the shift operation
            new_state = np.zeros_like(state, dtype=complex)

            for node in range(n_nodes):
                for side in [0, 1]:
                    amp = state[side*n_nodes + node]
```

```
                # Apply the Hadamard matrix based off the coin and new coin
                for new_coin in [0, 1]:
                    #calculate the amplitude of that coin side
                    coin_amp = H[new_coin, side] * amp

                    # Moves left if 0, right if 1
                    if new_coin == 0:
                        new_position = (node - 1) % n_nodes
                    else:
                        new_position = (node + 1) % n_nodes

                    # Update the new_state by adding the new coin amplitude
                    new_state[new_coin*n_nodes + new_position] += coin_amp

        state = new_state

        # Create a probability vector
        probabilities = np.zeros(n_nodes)
        for node in range(n_nodes):
            for side in [0, 1]:
                # square the ampltiudes in the state to get the probabilities; add to probability
                probabilities[node] += abs(state[side*n_nodes + node])**2

        # Measure to collapse the state to a position given the probabilities
        measurement = np.random.choice(range(n_nodes), p=probabilities)

    # Add the runtime once we reach the target node
    runtimes.append(steps)
return runtimes, np.mean(runtimes)
```

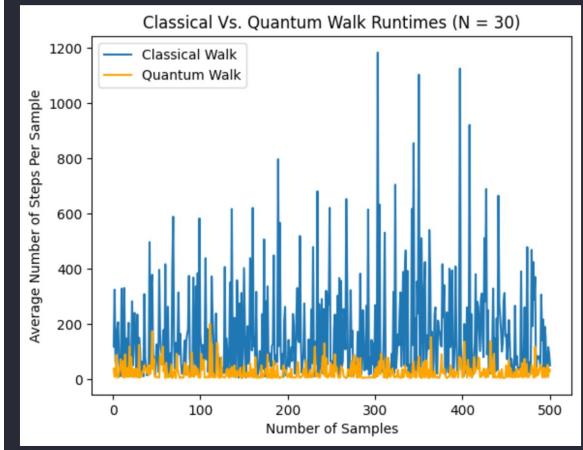
Nodes = 10: Classical Mean = 16.23, Quantum Mean = 5.37



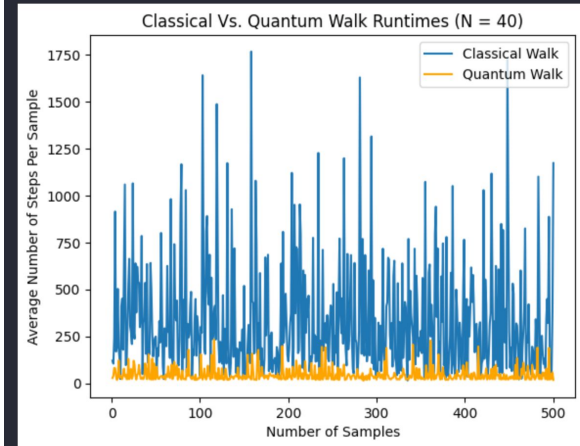
Nodes = 20: Classical Mean = 50.28, Quantum Mean = 23.47



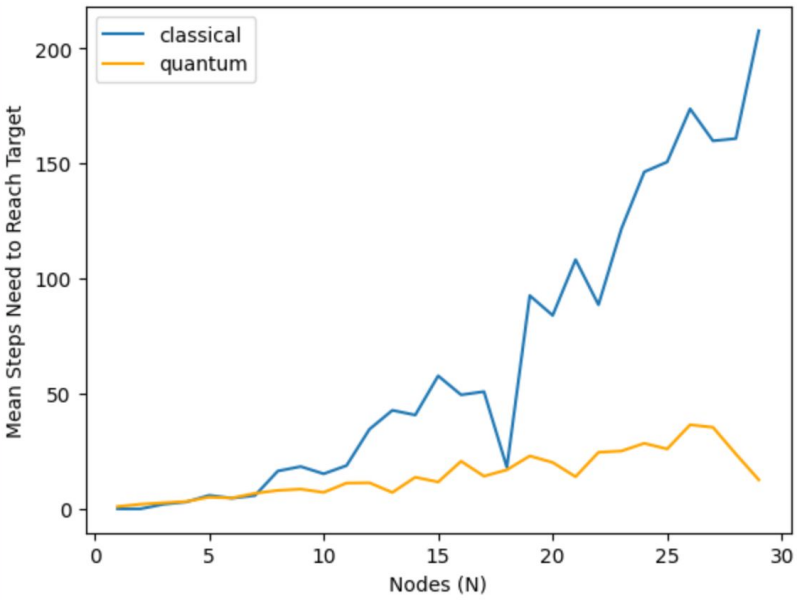
Nodes = 30: Classical Mean = 159.38, Quantum Mean = 28.72

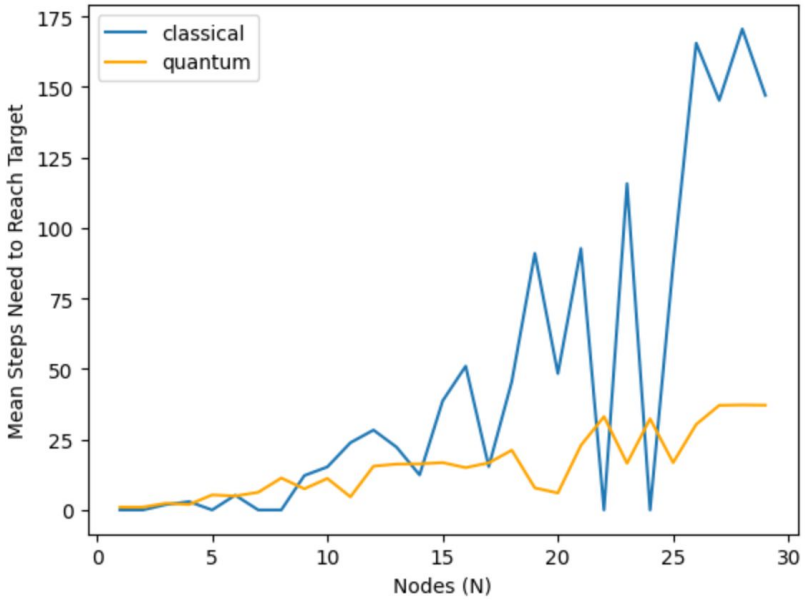


Nodes = 40: Classical Mean = 333.02, Quantum Mean = 46.09



Overall, quantum is a lot faster than classical!





Note: not the same N as as N steps in standard deviation